

Code Snippet

C# 코드의 해부학

키워드부터 논리 연산까지,
작동하는 코드의 시각적 토대 구축하기

런타임 환경: .NET | 모드: Blueprint | 상태: Ready

문장 (Statement)

표현식이 모여 완전한 명령을 이루는 단위.

```
Console.WriteLine(10 + 20 + 30 * 2);
```

표현식 (Expression)

값을 만들어내는 최소 단위의 코드.

종결자 (Terminator)

모든 문장의 끝을 알리는 필수 마침표. 없으면 컴파일 에러 발생.

시스템 예약어 (Keywords)

int void class
if bool break return public
as base byte catch char checked
const continue decimal default delegate
do double else enum event
explicit for for foreach finally fixed float
goto implicit if keyword new null object
internal is namespace override operator out
long out readonly ref private protected
sizeof stackalloc sbyte sealed short
string struct switch throw true try
typeof uint ulong unchecked unsafe
using virtual volatile while



C#이 미리 짰해둔 단어들.
식별자로 절대 사용할 수 없음.

사용자 생성 (Identifiers)

✓ 허용됨 (O)

alpha
_alpha
alpha10
Alpha

(알파벳, 숫자, 언더바 허용)

✗ 금지됨 (X)

break
273alpha
has space

(키워드, 숫자로 시작, 공백 포함 불가)

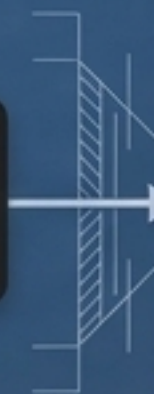
대소문자 관례 (Naming Conventions)

PascalCase

항상 대문자로 시작

CreateServer
Main

(클래스, 속성, 메서드)



i am a boy → IAmABoy

camelCase

소문자로 시작

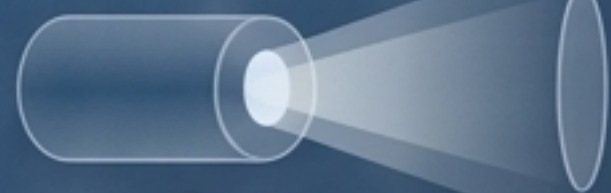
iLoveYou
age

(지역/전역 변수)



create server → createServer

변수 vs 메서드 판독기 (The Parentheses Detector)



() 감지
(Detects ())

WriteLine()

메서드 (Method)

괄호 없음
(No Parentheses)

PI

변수 (Variable)

Code Editor

```
// 이 줄은 기계가 읽지 않습니다.  
Console.WriteLine("Hello C#");  
  
/* 여러 줄을  
   숨길 때 사용합니다 */
```

주석은 프로그램의 실행에 영향을 주지 않는 투명 망토. 오직 WriteLine 안에 담긴 대상만 콘솔로 출력됩니다.

Terminal

Hello C#

C# 기본 자료형 마스터 테이블

자료형	예시	특징	핵심 연산
정수 (Integer)	273, -103	소수점 없음	+ - * / %
실수 (Real)	52.273, 0.0	소수점(.) 존재	사칙연산 (나머지 연산은 비권장)
문자 (Character)	'A', '가'	작은따옴표('), 단 하나의 글자	숫자로 변환될 수 있음
문자열 (String)	"Hello"	큰따옴표("), 문자의 집합	+ (문자열 연결)
불 (Bool)	true, false	오직 참과 거짓	논리 및 비교 연산

정수의 나눗셈 (Truncation)



정수끼리 연산하면 결과도 무조건 정수. 소수점 이하는 증발.

나머지 연산자 % (Modulo)

기본 및 음수 규칙

$$10 \% 5 = 0$$

$$7 \% 3 = 1$$

⚠ 음수의 나머지 연산 부호는
항상 '왼쪽 피연산자'를 따라감.

$$(-4) \% 3 = -1$$

DANGER ZONE

⚠ 위험 구간: 실수형 나머지

$$5.0 \% 2.2 = 0.6$$

💀 실수형 변수에 % 사용 비권장
(결과 예측 어려움)

Blackbox Laboratory: String vs Character Addition

실험 1: 문자열의 결합 (String)

"가"

"황"

+

[가황]

```
Console.WriteLine("가" + "황");
```

실험 2: 문자 덧셈의 배신 (Character)

'가'

'황'

'가' + '황'
(Unicode 44032) (Unicode 55203)

ERROR: 99235

```
Console.WriteLine('가' + '황');
```

작은따옴표(')로 묶인 문자는 내부적으로 숫자로 취급됩니다. 문자를 이어 붙으려면 반드시 큰따옴표(")를 사용하세요.

문자열의 해부학과 예외(Exception)



이스케이프 문자 (Escape Characters)

• X-ray transformation •

CODE
`\t`



OUTPUT

[수평 탭: 공백 4칸 띄우기]

CODE
`\n`



OUTPUT

[행 바꿈: Enter 키 효과]

CODE
`\"`



OUTPUT

[문자열 단기가 아닌 출력용
큰따옴표로 강제 변환]

CODE SNAPSHOT

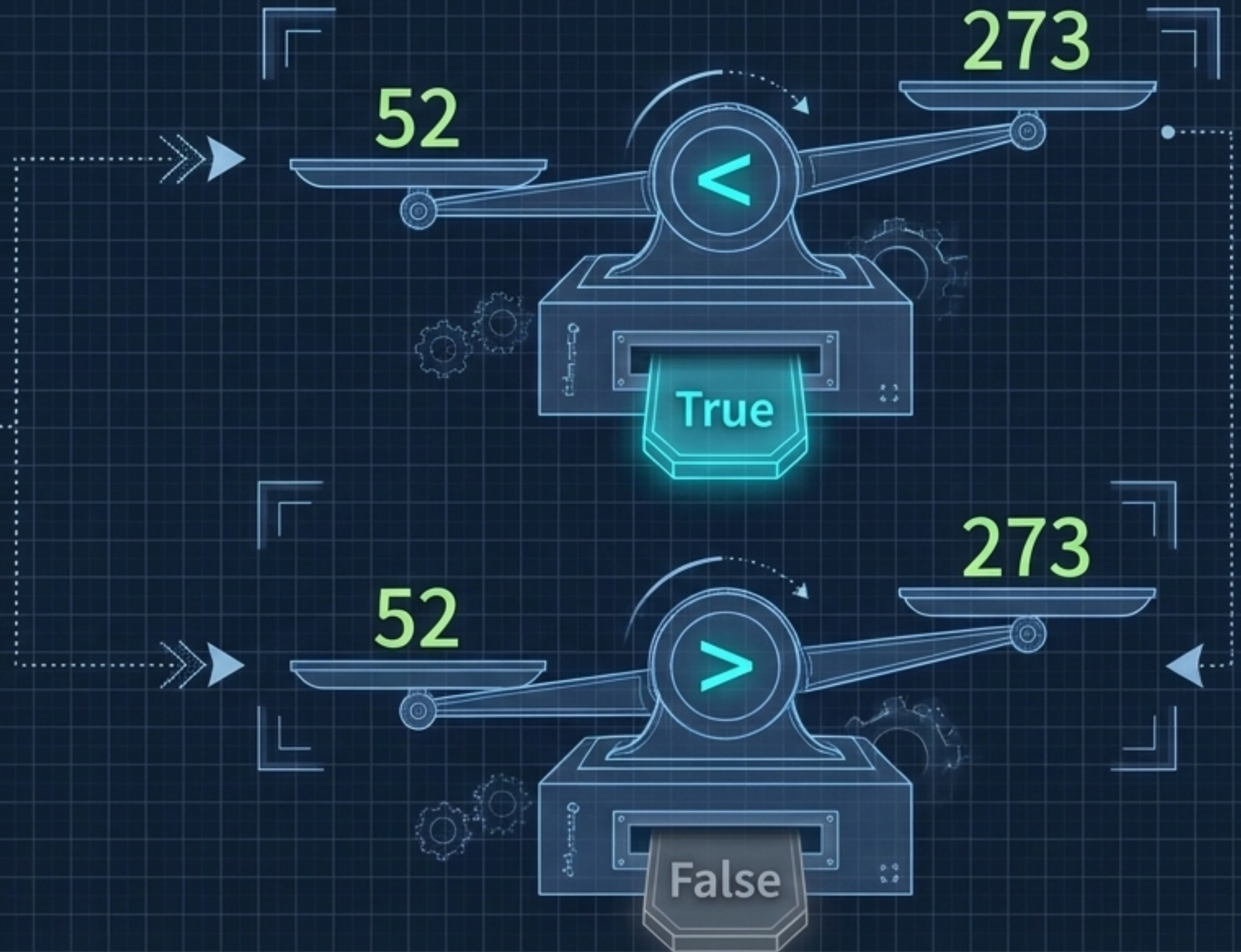
```
1 Console.WriteLine("\한빛\n아카데미");
```

RENDERED OUTPUT

"한빛"
"아카데미"

논리의 저울 (Boolean & Comparison)

== (같다)
!= (다르다)
<, >, <=, >= (크기 비교)



논리 연산자와 수직선 (Logical Operators & Number Line Models)

$3 < x < 8$

오류: C#은 연속된 범위 연산을 허용하지 않습니다.

논리합 || (OR) - 발산하는 화살표

$x < 3 \ || \ x > 8$



둘 중 하나만 참이어도 통과.

논리곱 && (AND) - 교차하는 교집합

$x > 3 \ \&\& \ x < 8$



두 범위가 완벽히 교차하는 부분만 선택.

논리의 조립 (Synthesis of Logic)



단순한 값(자료형)은 비교 연산을 통해 논리(Boolean)로 변환되고, 논리 연산자를 통해 복잡한 비즈니스 규칙(범위)을 형성하며, 종속자(문장)가 되어 마침내 결과를 출력합니다. 이것이 C# 프로그래밍 사고의 기초입니다.